6
Fine-Tuning with Preference Alignment

Supervised Fine-Tuning (SFT) has been crucial in adapting LLMs to perform specific tasks. However, SFT struggles to capture the nuances of human preferences and the long tail of potential interactions that a model might encounter. This limitation has led to the development of more advanced techniques for aligning AI systems with human preferences, grouped under the umbrella term *preference alignment*.

Preference alignment addresses the shortcomings of SFT by incorporating direct human or AI feedback into the training process. This method allows a more nuanced understanding of human preferences, especially in complex scenarios where simple supervised learning falls short. While numerous techniques exist for preference alignment, this chapter will primarily focus on **Direct Preference Optimization (DPO)** for simplicity and efficiency.

In this chapter, we will talk about the type of data that is required by preference alignment algorithms like DPO. We will build our own dataset to modify the writing style of our model, making it less artificial and more authentic. We will introduce the DPO algorithm and implement it to align the model trained in *Chapter 5*.

In this chapter, we will cover the following topics:

- Understanding preference datasets
- How to create our own preference dataset
- **Direct preference optimization (DPO)**
- Implementing DPO in practice to align our model

By the end of this chapter, you will be able to create your own preference datasets and align models with diverse techniques.

All the code examples from this chapter can be found on GitHub at https://github.com/PacktPublishing/LLM-Engineering.

Understanding preference datasets

The principles for creating high-quality preference datasets are the same as those discussed in *Chapter 5* for instruction datasets. We want to maximize the accuracy, diversity, and complexity of our samples. To achieve this, we follow the same stages, as outlined in *Figure 6.1*: data curation, deduplication, decontamination, quality evaluation, exploration, generation, and augmentation.

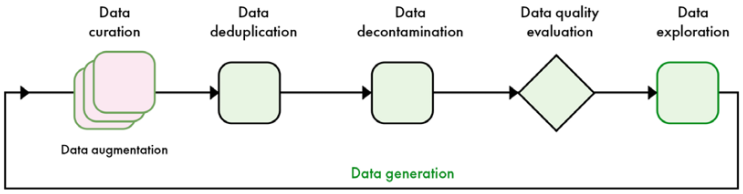


Figure 6.1 – Overview of the post-training data pipeline covered in this chapter

To avoid repetition, this section will focus on the main differences between instruction and preference datasets. We will introduce the structure of preference samples and the ideal size for preference datasets. Then, we will focus on the two stages that differ most from creating instruction datasets: data generation and evaluation.

Preference data

Preference datasets lack the standardization of instruction datasets due to varying data requirements across different training algorithms. Preference data comprises a collection of responses to a given instruction, ranked by humans or language models. This chapter focuses on DPO, so we will examine the specific data format required by this algorithm.

As illustrated in *Table 6.1*, the structure of DPO datasets is straightforward: each instruction is paired with one preferred answer and one rejected answer. The objective is to train the model to generate the preferred response rather than the rejected one.

Instruction	
Tell me a joke about octopuses.	
Chosen answer	**Rejected answer**
Why don't octopuses play cards in casinos? Because they can't count past eight.	How many tickles does it take to make an octopus laugh? Ten tickles.

Table 6.1 – Example of sample from the mlabonne/orpo-dpo-mix-40k dataset

In preference datasets, the rejected response is as important as the chosen one. Without the rejected response, the dataset would be a simple instruction set. Rejected responses represent the behavior we aim to eliminate from the model. This provides a lot of flexibility and allows us to use preference datasets in many contexts. Here is a list of examples where preference datasets are more beneficial to use compared to using SFT alone:

- **Chatbots**: In conversational AI, the quality of responses often depends on subjective factors like naturalness, engagement, and contextual appropriateness. A preference dataset allows the model to learn these nuanced aspects by comparing better and worse responses. Simple SFT might not capture the subtleties of what makes one response preferable over another in a given context.
- **Content moderation**: Determining whether content is appropriate or violates guidelines often involves nuanced judgments. Preference

datasets can help the model learn to distinguish between borderline cases by comparing examples of content that is and isn't acceptable. This is more effective than binary classification through SFT, as it helps the model understand the reasoning behind moderation decisions.

- **Summarization**: The quality of a summary often depends on factors like conciseness, relevance, and coherence. By using preference datasets, models can learn to generate summaries that humans find more useful and informative. Simple SFT might result in summaries that are technically correct but less preferable to human readers.
- **Code generation**: In coding tasks, there are often multiple correct solutions, but some are more efficient or readable, or follow better practices than others. Preference datasets can help the model learn these qualitative aspects of code quality, which might not be captured by simple correctness-based SFT.
- **Creative writing**: For tasks like story generation or poetry writing, the quality of the output is highly subjective and multifaceted. Preference datasets can capture human judgments about style, creativity, and emotional impact better than instruction datasets, which might focus more on technical correctness or adherence to prompts.
- **Translation**: While traditional metrics like BLEU scores can measure translation accuracy, they don't always capture the fluency or naturalness of the translation. Preference datasets can help models learn to produce translations that native speakers prefer, even when multiple translations are technically correct.

In all these scenarios, preference datasets enable a more refined training approach. They capture subjective quality assessments and human preferences that extend beyond simple correctness or adherence to instructions. This method can produce models that generate output that is not only technically accurate but also better aligned with human judgment and preferences in complex, open-ended tasks.

Unlike instruction datasets, there are no standardized storage formats like Alpaca or ShareGPT. Most preference datasets follow a structure similar to that shown in *Table 6.1*, with columns for an instruction, a preferred answer, and a rejected answer. Multi-turn conversations are uncommon in preference alignment. At the time of writing, major fine-tuning libraries do not support multi-turn conversations and typically extract only the first or last message in a conversation.

Data quantity

DPO datasets typically require fewer samples than instruction datasets to significantly impact model behavior. As with instruction datasets, the required sample count depends on model size and task complexity. Larger models are more sample-efficient and thus require less data, while complex tasks demand more examples to capture the desired behavior. Once again, data quality is crucial, and a large number of preference pairs is generally beneficial.

General-purpose alignment is used by LLM providers to improve the overall performance of the fine-tuned models. This requires preference datasets with millions of samples. Major players in the AI industry, including Nvidia and Meta, are converging on similar post-training pipelines, involving multiple rounds of preference alignment, and extensive use of synthetic data. This consensus suggests that these methods are

proving to be the most effective for pushing the boundaries of language model capabilities.

On a smaller scale, the open-source community uses datasets ranging from 10,000 to 100,000 samples to enhance model performance. This approach has proven effective not only in improving benchmark scores but also in healing networks after merging, pruning, and other modifications. Generally, DPO is less destructive than SFT and has a milder impact on the final model.

On the other hand, tasks like the ones previously described require fewer preference pairs. Task-specific alignment focuses on improving model performance for a particular function, such as modifying the writing style, refusing certain instructions, and so on. These alignments can often be achieved with smaller datasets, ranging from 100 to 10,000 preference pairs, depending on the task's complexity.

An example of an application that requires few samples is instructing the model to state that it wasn't trained by OpenAI, Meta, or another LLM provider. This can be achieved using a preference dataset, where the rejected answers are those claiming alternative origins, and the chosen answers are responses where the model correctly states that it was trained by you. A relatively small dataset of 200 to 500 pairs can be enough for this task.

Data generation and evaluation

When creating preference datasets, data generation and evaluation are closely linked. We first create answers and then rate them to make the final dataset. In the following, we introduce both steps as one process instead of two separate ones.

Generating preferences

Before making new preference data, it's good to look at relevant open-source datasets. There are fewer of these compared to instruction datasets, but you can find high-quality preference datasets on the Hugging Face Hub. These can be used for specific tasks or to add to your own dataset. Well-known preference datasets include the Anthropic HH-RLHF dataset, which has human preferences for helpful and harmless AI responses, and the OpenAI Summarize from Human Feedback dataset, which focuses on article summaries.

DPO datasets can be created using various methods, each with its own trade-offs between quality, cost, and scalability. These methods can be tailored to specific applications and require varying degrees of human feedback. We divide them into four main categories:

- **Human-generated, human-evaluated datasets**: This method involves hiring people to both create responses to prompts and evaluate the quality of these responses. While this approach can capture nuanced human preferences and is ideal for complex tasks, it's extremely resource-intensive and difficult to scale. As a result, it's primarily used by large AI companies with substantial resources.
- **Human-generated, LLM-evaluated datasets**: This method can be useful if you have a lot of existing human-generated content. However, it's rarely used in practice due to inefficiency, as it still re-

quires significant human input for response generation while potentially missing nuanced preferences during the LLM evaluation stage.

- **LLM-generated, human-evaluated datasets**: This method offers a good balance between quality and efficiency. LLMs generate multiple responses to prompts, and humans rank these responses. This approach is often preferred because humans are generally better at judging answers than writing them from scratch. It allows the rapid generation of diverse responses while still capturing human preferences effectively. However, it may not provide creative or unexpected responses that humans might generate.
- **LLM-generated, LLM-evaluated datasets**: Fully synthetic datasets, where both generation and evaluation are done by LLMs, are becoming increasingly common due to their scalability and cost-effectiveness. This method can produce massive datasets quickly and improves as LLM capabilities advance. However, it requires careful prompt engineering to ensure quality and diversity, and may perpetuate biases or limitations of the generating LLM.

In practice, human-generated datasets are expensive, difficult to scale, and not necessarily of the highest quality. On the other hand, human evaluation is quite valuable but can be difficult to scale, which is why large datasets benefit from LLM evaluation. In addition to these high-level considerations, the way you obtain your data and how you plan to use it also need to be considered. For example, applications with many users can embed a feedback mechanism to provide preferences. This can be as simple as a `like` and `dislike` score, or something more in-depth with text.

Note that evaluation is not always required and preferences can emerge naturally from the generation process. For instance, it is possible to use a high-quality model to generate preferred outputs and a lower-quality or intentionally flawed model to produce less preferred alternatives. This creates a clear distinction in the preference dataset, allowing more effective training of AI systems to recognize and emulate high-quality outputs. The `Intel/orca_dpo_pairs` dataset available on the Hugging Face Hub was created with this process.

Another approach is to compare model-generated outputs with human-written responses, which can provide insights into how well the model aligns with actual human preferences and highlight areas where the model may be lacking. This can be used to copy a particular style and give a more authentic tone to the model.

Tips for data generation

The data generation is consistent between instruction and preference datasets. Prompts should be designed to encourage diversity and complexity in the model's responses. By crafting prompts that explicitly request different approaches or styles, we can ensure a wide range of outputs that capture the varied nature of human preferences.

For instance, when generating summaries, one might request variations such as concise summaries, detailed summaries, and summaries focusing on key points. This approach not only produces a diverse dataset but also helps in understanding how different styles and approaches align with human preferences.

Introducing variability in the outputs is another crucial aspect of generating synthetic preference datasets. This can be achieved by manipulating the temperature settings or employing other sampling methods in the LLM. Higher temperature settings tend to produce more creative and diverse responses, while lower settings result in more focused and deterministic outputs. This creates a trade-off between diversity and coherence, which depends on the kind of data we want to generate. For example, generating code requires low creativity, thus low temperature, while writing articles can be high temperature.

Using multiple LLMs to generate samples can be better than using just one model. Some LLMs are better at specific tasks, and this approach also adds more variety. This approach is used by popular open-source datasets like `argilla/Capybara-Preferences`, combining GPT-4 with open-weight models. The evaluation process then selects the chosen and the rejected answers.

Evaluating preferences

Data evaluation can be performed by human raters or automated with LLMs. **LLM evaluation** involves developing detailed criteria, creating a prompt that clearly communicates these guidelines to the LLM, and using the model to select preferred and rejected responses. While more scalable than human rating and allowing the consistent application of criteria, this quality of LLM evaluation depends directly on the model's performance and the provided guidelines. It may miss subtle human preferences or cultural nuances. However, as LLMs continue to improve, their ability to make nuanced judgments improves as well, potentially leading to higher-quality datasets over time.

Implementing LLM evaluation for preference datasets can be done through absolute scoring or pairwise ranking. In absolute scoring, the LLM assigns a numerical score or categorical rating to each response based on predefined criteria. This method is straightforward but may suffer from inconsistency across different prompts or evaluation sessions. Pairwise ranking, on the other hand, involves presenting the LLM with two responses and asking it to choose the better one or rank them. This approach more closely mimics the format of human evaluation and can lead to more consistent results.

For absolute scoring, you would create a prompt that outlines the evaluation criteria and asks the LLM to rate the response on a specific scale (e.g., 1-5 or poor/fair/good/excellent). The prompt might look like this: "Rate the following response on a scale of 1-5 based on relevance, coherence, and helpfulness: [`INSERT RESPONSE`]." For pairwise ranking, the prompt could be: "Compare the following two responses. Which one is better in terms of relevance, coherence, and helpfulness? Response A: [`INSERT RESPONSE A`] Response B: [`INSERT RESPONSE B`]."

The comparative nature of preference datasets makes pairwise ranking an ideal approach for evaluation. This method is generally more accurate and more closely correlated to human judgment than absolute scoring. Pairwise ranking mimics the natural way humans compare options, making it easier for both human raters and LLMs to provide consistent and meaningful evaluations.

We can further improve the accuracy of pairwise ranking by providing a ground-truth answer and using chain-of-thought reasoning. This approach encourages the evaluating LLM to consider multiple aspects of the responses and articulate its decision-making process, leading to more thorough and justified evaluations. When no ground-truth answer is available, we can prompt the LLM to create a grading note, which is a description of the expected answer. This technique works particularly well in scenarios where the LLM doesn't have extensive knowledge about a given topic, as it forces the model to establish clear criteria for evaluation before assessing the responses.

Here's a concrete implementation of an LLM-as-a-judge prompt to perform pairwise ranking:

Instruction
You are an answer judge. Your goal is to compare answer A and answer B. I want to know which answer does a better job of answering the instruction in terms of relevance, accuracy, completeness, clarity, structure, and conciseness.
Instruction: {instruction}
Answer A: {answer_a}
Answer B: {answer_b}
Explain your reasoning step by step and output the letter of the best answer using the following structure:
Reasoning: (compare the two answers)
Best answer: (A or B)

Table 6.2 – Example of LLM-as-a-judge prompt for pairwise ranking with one instruction and two answers

However, it's important to note that LLM-based evaluation can be subject to several types of bias:

- **Position bias**: In relative scoring, LLM judges tend to favor the first answer presented. This bias can skew results and lead to inaccurate preferences.
- **Length bias**: Similar to humans, LLM judges often show a preference for longer answers, potentially overlooking the quality of shorter, more concise responses.
- **Family bias**: LLM judges may favor responses that are generated by themselves or models from the same family, potentially due to similarities in language patterns or knowledge bases.

To mitigate these biases and enhance the quality of preference datasets, several solutions can be implemented. One key approach is to randomize the order of answer A and answer B in each comparison, which can counteract position bias by ensuring that the order of presentation doesn't consistently influence the evaluation. Another valuable strategy involves providing few-shot examples that demonstrate a balanced distribution of scores. These examples serve to calibrate the judge LLM's internal scoring mechanism and can effectively address both length and family bias by illustrating that shorter answers or those from different model families can also be of high quality. Additionally, employing multiple models as a jury, rather than relying on a single LLM judge, can significantly improve the robustness of the evaluation process. This multi-model approach helps to balance out individual biases that may be present in any single

model, leading to a more comprehensive and accurate assessment of the responses.

In the next section, we will create our own preference dataset. We will rely on the data generation process to naturally create chosen (human-generated) and rejected (LLM-generated) answers.

Creating our own preference dataset

Our model can currently write paragraphs about topics related to machine learning, but it doesn't have the same writing style as the original authors. This is a typical use case for preference alignment, where we want to change the "voice" of the model to closely imitate the source data. It's important to note that, experimentally, DPO tends to make models more verbose and pushes them to use very formal language. Therefore, the training will need to use DPO surgically to avoid this pitfall and instead adopt the less formal style of these blog articles.

In this section, we will create a preference dataset where the chosen answers are extracts from the text, while rejected answers are generated by the model. To implement it, we will modify the code created in *Chapter 5*, which was designed to generate instruction datasets.

As seen in the previous section, preference and instruction datasets rely on the same principles. Instead of pairs of instructions and answers, we need triples (instruction, answer 1, answer 2). What's interesting in this setting is that we have ground-truth answers in the text chunks, which means we don't need complex evaluation processes like LLM judges. To make sure that these extracts are high-quality, we will implement two additional quality filters, based on length and punctuation. *Figure 6.2* summarizes the end-to-end process:

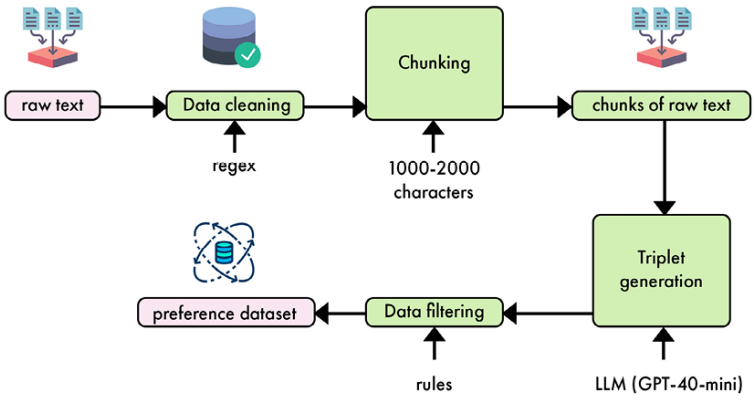


Figure 6.2 – Synthetic data generation pipeline from raw text to preference dataset

We are now ready to implement the preference data generation pipeline:

1. We start by importing the necessary libraries.

```
import concurrent.futures
import json
import re
from typing import List, Tuple
from datasets import Dataset
```

```
from openai import OpenAI
from tqdm.auto import tqdm
```

2. Instead of the `InstructionAnswerSet` class, we now have a `PreferenceSet` class. This class is designed to handle triples of instructions, generated answers (rejected), and extracted answers (chosen).

```
class PreferenceSet:
    def __init__(self, triples: List[Tuple[str, str, str]]):
        self.triples = triples
    @classmethod
    def from_json(cls, json_str: str) -> 'PreferenceSet':
        data = json.loads(json_str)
        triples = [(triple['instruction'], triple['generated_answer'], triple['e
                    for triple in data['preference_triples']]
        return cls(triples)
    def __iter__(self):
        return iter(self.triples)
```

3. The `load_articles_from_json`, `clean_text`, and `extract_substrings` functions remain unchanged from the original code. Let's start with `load_articles_from_json`, which takes our JSON file (`cleaned_documents.json`) containing the articles as input and returns a Hugging Face dataset with the text and metadata (ID, platform, author ID, author full name, link).

```
def load_articles_from_json(file_path: str) -> Dataset:
    with open(file_path, "r") as file:
        data = json.load(file)
    return Dataset.from_dict(
        {
            "id": [item["id"] for item in data["artifact_data"]],
            "content": [item["content"] for item in data["artifact_data"]],
            "platform": [item["platform"] for item in data["artifact_data"]],
            "author_id": [item["author_id"] for item in data["artifact_data"]],
            "author_full_name": [item["author_full_name"] for item in data["arti
            "link": [item["link"] for item in data["artifact_data"]],
        }
    )
```

4. The `clean_text` function removes non-alphanumeric characters except for apostrophes, periods, commas, exclamation marks, and question marks. It also replaces multiple whitespaces with a single space to ensure proper formatting.

```
def clean_text(text: str) -> str:    text = re.sub(r"[^\w\s.,!?']", " ", text)
    return text.strip()
```

5. The `extract_substrings` function splits articles into chunks with a length between 1,000 and 2,000 characters. To make sure that the splitting doesn't break sentences, which could modify their meanings, we use a regex to only split after the end of a sentence.

```
def extract_substrings(dataset: Dataset, min_length: int = 1000, max_length: int
    extracts = []
    sentence_pattern = r"(?<!\w\.\w.)(?<![A-Z][a-z]\.)(?<=\.|\?|\!)\s"
    for article in dataset["content"]:
```

```
        cleaned_article = clean_text(article)
        sentences = re.split(sentence_pattern, cleaned_article)
        current_chunk = ""
        for sentence in sentences:
            sentence = sentence.strip()
            if not sentence:
                continue
            if len(current_chunk) + len(sentence) <= max_length:
                current_chunk += sentence + " "
            else:
                if len(current_chunk) >= min_length:
                    extracts.append(current_chunk.strip())
                current_chunk = sentence + " "
        if len(current_chunk) >= min_length:
            extracts.append(current_chunk.strip())
    return extracts
```

6. The `generate_preference_triples` function replaces the original `generate_instruction_answer_pairs` function. The prompt is adapted from the instruction version and is designed to generate triples instead of pairs. It also provides general guidance about the type of instructions we're interested in, how to extract answers from articles, and how to style them:

```
def generate_preference_triples(extract: str, client: OpenAI) -> List[Tuple[str,
    prompt = f"""Based on the following extract, generate five instruction-answe
1. An instruction asking about a specific topic in the context.
2. A generated answer that attempts to answer the instruction based on the conte
3. An extracted answer that is a relevant excerpt directly from the given contex
Instructions must be self-contained and general, without explicitly mentioning a
Important:
- Ensure that the extracted answer is a verbatim copy from the context, includin
- Do not add any ellipsis (...) or [...]  to indicate skipped text in the extrac
- If the relevant text is not continuous, use two separate sentences from the co
Provide your response in JSON format with the following structure:
{{
    "preference_triples": [
        {{
            "instruction": "...",
            "generated_answer": "...",
            "extracted_answer": "..."
        }},
        ...
    ]
}}
    Extract:
    {extract}
"""
```

7. In the same function, we use GPT-4o-mini to generate our answers using JSON mode. We specify in the system prompt that we want triples instead of pairs. The JSON answers are directly parsed by our `PreferenceSet` class to return the expected list of tuples.

```
    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": "You are a helpful assistant who generates instructio
            },
            {"role": "user", "content": prompt},
```

```
            ],
            response_format={"type": "json_object"},
            max_tokens=2000,
            temperature=0.7,
        )
        result = PreferenceSet.from_json(completion.choices[0].message.content)
        return result.triples
```

8. Two new filtering functions are introduced for the preference data pipeline: `filter_short_answers` and `filter_answer_format`. These functions filter out short answers and ensure that answers start with an uppercase letter and end with proper punctuation. We use them as heuristics to filter out samples with poor quality.

```
def filter_short_answers(dataset: Dataset, min_length: int = 100) -> Dataset:
    def is_long_enough(example):
        return len(example['chosen']) >= min_length
    return dataset.filter(is_long_enough)
def filter_answer_format(dataset: Dataset) -> Dataset:
    def is_valid_format(example):
        chosen = example['chosen']
        return (len(chosen) > 0 and
                chosen[0].isupper() and
                chosen[-1] in ('.', '!', '?'))
    return dataset.filter(is_valid_format)
```

9. The `create_preference_dataset` function replaces the original `create_instruction_dataset` function. This function now works with triples instead of pairs and uses different column names in the resulting dataset.

```
def create_preference_dataset(dataset: Dataset, client: OpenAI, num_workers: int
    extracts = extract_substrings(dataset)
    preference_triples = []
    with concurrent.futures.ThreadPoolExecutor(max_workers=num_workers) as execu
        futures = [
            executor.submit(generate_preference_triples, extract, client)
            for extract in extracts
        ]
        for future in tqdm(concurrent.futures.as_completed(futures), total=len(f
            preference_triples.extend(future.result())
    instructions, generated_answers, extracted_answers = zip(*preference_triples
    return Dataset.from_dict(
        {
            "prompt": list(instructions),
            "rejected": list(generated_answers),
            "chosen": list(extracted_answers)
        }
    )
```

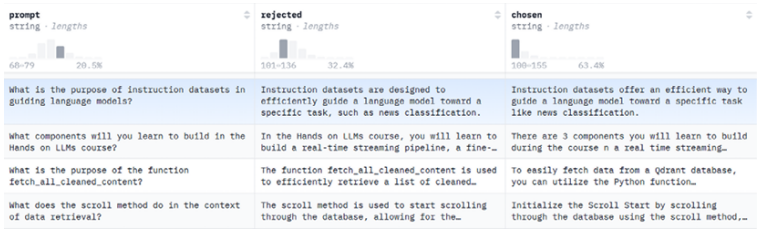
10. The main function is updated to include the new filtering steps and to use the preference dataset creation function:

```
def main(dataset_id: str) -> Dataset:
    client = OpenAI()
    # 1. Load the raw data
    raw_dataset = load_articles_from_json("cleaned_documents.json")
    print("Raw dataset:")
    print(raw_dataset.to_pandas())
    # 2. Create preference dataset
    dataset = create_preference_dataset(raw_dataset, client)
```

```
    print("Preference dataset:")
    print(dataset.to_pandas())
    # 3. Filter out samples with short answers
    dataset = filter_short_answers(dataset)
    # 4. Filter answers based on format
    dataset = filter_answer_format(dataset)
    # 5. Export
    dataset.push_to_hub(dataset_id)
    return dataset
```

The `create_preference_dataset()` function generated 2,970 samples. This dataset is then heavily filtered to only retain 1,467 samples by removing answers that are too short or not properly formatted (for example, answers that start with an uppercase letter or end with a period, exclamation mark, or question mark).

The final dataset is available on the Hugging Face Hub at the following address: https://huggingface.co/datasets/mlabonne/llmtwin-dpo. You can see in *Figure 6.3* an example that captures a subtle nuance in terms of writing style. Both answers are correct, but the **chosen** (extracted) answer sounds slightly more casual.



prompt string · lengths	rejected string · lengths	chosen string · lengths
60–79 20.5%	101–136 32.4%	100–155 63.4%
What is the purpose of instruction datasets in guiding language models?	Instruction datasets are designed to efficiently guide a language model toward a specific task, such as news classification.	Instruction datasets offer an efficient way to guide a language model toward a specific task like news classification.
What components will you learn to build in the Hands on LLMs course?	In the Hands on LLMs course, you will learn to build a real-time streaming pipeline, a fine-…	There are 3 components you will learn to build during the course n a real time streaming…
What is the purpose of the function fetch_all_cleaned_content?	The function fetch_all_cleaned_content is used to efficiently retrieve a list of cleaned…	To easily fetch data from a Qdrant database, you can utilize the Python function…
What does the scroll method do in the context of data retrieval?	The scroll method is used to start scrolling through the database, allowing for the…	Initialize the Scroll Start by scrolling through the database using the scroll method,…

Figure 6.3 – Screenshot of the mlabonne/llmtwin-dpo preference dataset on the Hugging Face Hub

To produce this dataset, we iterated many times over the prompt to generate the data. This required some manual evaluation and experiments until we reached satisfying results. The quality of the prompt is fundamental in this process, which is why it is recommended to follow a similar process to generate your own preference datasets.

In the next section, we will introduce concepts related to **Reinforcement Learning from Human Feedback** (**RLHF**) and DPO. This will cover new parameters and ideas that are implemented in the final section of this chapter.

Preference alignment

Preference alignment regroups techniques to fine-tune models on preference data. In this section, we provide an overview of this field and then focus on the technique we will implement: **Direct Preference Optimization** (**DPO**).

Reinforcement Learning from Human Feedback

Reinforcement Learning from Human Feedback (**RLHF**) combines **reinforcement learning** (**RL**) with human input to align models with human preferences and values. RLHF emerged as a response to challenges in traditional RL methods, particularly the difficulty of specifying reward

functions for complex tasks and the potential for misalignment between engineered rewards and intended objectives.

The origins of RLHF can be traced back to the field of **preference-based reinforcement learning** (**PbRL**), which was independently introduced by Akrour et al. and Cheng et al. in 2011. PbRL aimed to infer objectives from qualitative feedback, such as pairwise preferences between behaviors, rather than relying on quantitative reward signals. This approach addressed some of the limitations of conventional RL, where defining appropriate reward functions can be challenging and prone to reward hacking or unintended behaviors.

The term RLHF was coined later, around 2021-2022, as the approach gained prominence in the context of training LLMs. However, the core ideas had been developing for years prior. A seminal paper by Christiano et al. in 2017 demonstrated the effectiveness of learning reward models from human preferences and using them to train RL agents. This work showed that RLHF could match or exceed the performance of agents trained on hand-engineered rewards, but with significantly less human effort.

At its core, RLHF works by iteratively improving both a reward model and a policy:

- **Reward model learning**: Instead of using a pre-defined reward function, RLHF learns a reward model from human feedback. This is typically done by presenting humans with different answers and asking them to indicate which one they prefer. These preferences are used to train a reward model, often using a Bradley-Terry model or similar approaches that map preferences to underlying utility functions.
- **Policy optimization**: With the learned reward model, standard RL algorithms can be used to optimize a policy. This policy generates new behaviors that aim to maximize the predicted rewards from the learned model.
- **Iterative improvement**: As the policy improves, it generates new behaviors that can be evaluated by humans, leading to refinements in the reward model. This cycle continues, ideally resulting in a policy that aligns well with human preferences.

A key innovation in RLHF is its approach to handling the high cost of human feedback. Rather than requiring constant human oversight, RLHF allows for asynchronous and sparse feedback.

The learned reward model serves as a proxy for human preferences, enabling the RL algorithm to train continuously without direct human input for every action.

As an example, *Figure 6.4* shows a high-level view of the **Proximal Policy Optimization** (**PPO**) algorithm, which is one of the most popular RLHF algorithms. Here, the reward model is used to score the text that is generated by the trained model. This reward is regularized by an additional **Kullback–Leibler** (**KL**) divergence factor, ensuring that the distribution of tokens stays similar to the model before training (frozen model).

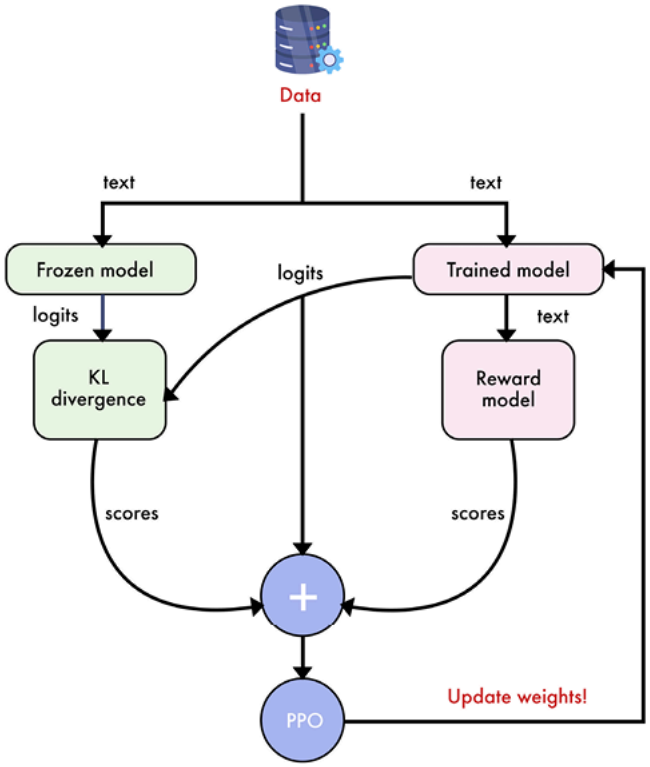


Figure 6.4 – High-level view of the PPO algorithm for preference alignment

While RLHF has proven effective for aligning AI systems with human preferences, it faces challenges due to its iterative nature and reliance on a separate reward model, which can be computationally expensive and potentially unstable. Despite theoretical superiority, RLHF algorithms have also experimentally underperformed compared to simpler approaches. One such approach that has gained significant attention is DPO.

Direct Preference Optimization

Introduced by Rafailov et al. in their 2023 paper *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*, DPO offers a streamlined alternative to traditional RLHF methods.

DPO's core innovation lies in its reformulation of the preference learning problem. Unlike RLHF, which typically involves training a separate reward model and then using reinforcement learning algorithms like PPO to fine-tune the language model, DPO takes a more direct approach.

It derives a closed-form expression for the optimal policy under the standard RLHF objective of maximizing expected reward subject to a KL-divergence constraint with a reference policy. This mathematical insight allows DPO to express the preference learning problem directly in terms of the policy, eliminating the need for a separate reward model or complex reinforcement learning algorithms.

In practical terms, DPO can be implemented as a simple binary cross-entropy loss function that operates directly on the language model's output probabilities. This loss function encourages the model to assign higher probability to preferred responses and lower probability to non-preferred responses, while maintaining closeness to a reference (frozen) model. The importance of the reference model is directly controlled via a beta parameter between 0 and 1. The reference model is ignored when

beta is equal to 0, which means that the trained model can be very different from the SFT one. In practice, a value of 0.1 is the most popular one, but this can be tweaked, as we'll see in the next section.

The simplicity of this approach allows optimization using standard gradient descent techniques, without the need for sampling from the model during training or implementing complex RL algorithms. *Figure 6.5* shows a high-level view of the DPO algorithm, greatly simplifying the training process compared to *Figure 6.4*.

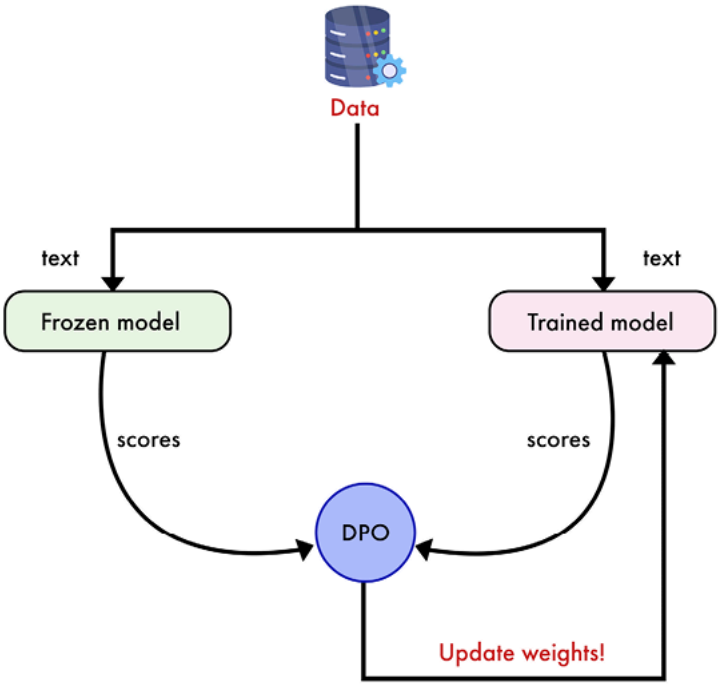


Figure 6.5 – High-level view of the DPO algorithm for preference alignment

DPO has several advantages over traditional RLHF methods. As previously mentioned, it significantly simplifies the preference learning pipeline, reducing the engineering complexity associated with RLHF methods. By eliminating the need for a separate reward model and RL algorithms, DPO is more computationally efficient than traditional RLHF approaches. Particularly when trained with adapters (LoRA, QLoRA), the frozen and trained models don't have to be separated. Indeed, since we're only training adapters, the trained model is not modified. This allows us to only load one model instead of two, which saves additional VRAM.

Despite its simplicity, DPO often matches the performance of more complex RLHF methods. It also tends to be more stable during training and less sensitive to hyperparameters. The simplified approach makes DPO easier to implement and scale, particularly for small teams without extensive RL knowledge.

While RLHF allows iterative improvement through multiple training rounds and can dynamically adapt to new preferences, DPO offers a more straightforward path to achieving similar results. The choice between DPO and PPO-based RLHF often comes down to a trade-off between ease of implementation and potential peak performance. For large-scale training runs with millions of preference samples, PPO-inspired methods still have a higher performance ceiling. However, for most applications, DPO

provides the majority of the performance benefits at a lower computational and engineering cost.

Both RLHF and DPO benefit significantly from the integration of synthetic data. As LLMs become more capable, they can generate data that surpasses human-created content in quality and diversity. This enables a virtuous cycle where better models produce better training data, which in turn leads to further model improvements. The iterative nature of both approaches allows multiple rounds of model refinement, each focusing on different aspects of model performance and gradually enhancing capabilities across various domains.

Despite its advantages, DPO is not without drawbacks. Like RLHF, DPO still requires paired preference data, which can be expensive and time-consuming to collect. DPO lacks some of the theoretical guarantees associated with reinforcement learning approaches. There may be scenarios where the added flexibility of RLHF is beneficial, particularly for complex tasks or environments.

Nonetheless, DPO is ideal in most cases, including our twin LLM example. In the next section, we will implement it using Unsloth.

Implementing DPO

In this section, we will DPO fine-tune the **TwinLlama-3.1-8B** model we created in *Chapter 5*. For ease of use and to maximize performance, we will again use the Unsloth library for our DPO implementation. Depending on the available VRAM, you can choose between LoRA (higher quality, speed, and VRAM usage) and QLoRA (lower quality, speed, and VRAM usage). This technique, along with other preference alignment algorithms, is also available in TRL and Axolotl.

This example can be seen as an advanced application of DPO. Indeed, our objective of imitating a writing style conflicts with the natural tendency of DPO to encourage formal language. This is partly due to the fact that chosen answers are often more formal than rejected ones. In practice, this will force us to do light fine-tuning, with a low learning rate and number of epochs. To find the best hyperparameters, we trained over 20 models and compared their outputs on a set of questions, including "Write a paragraph to introduce supervised fine-tuning." This allowed us to select the model and parameters that worked best for this task.

The dependencies are the same as those in *Chapter 5* with SFT and can be found in the book's GitHub repository (https://github.com/Packt-Publishing/LLM-Engineering) or in Unsloth's repo (https://github.com/unslothai/unsloth):

1. First, we want to access a gated model and (optionally) upload our fine-tuned model to Hugging Face (https://huggingface.co/). This requires us to log in to an account. If you don't have an account, you can create one and store your API key (**Settings** | **Access Tokens** | **Create new token**) in the `.env` file:

```
HF_TOKEN = YOUR_API_KEY
```

2. Make sure that your Comet ML API key is also in the `.env` file. Otherwise, the code will crash and raise an error when training starts.

```
COMET_API_KEY = YOUR_API_KEY
```

3. Before we import all the necessary packages, we want to apply a patch for the `DPOTrainer` class from TRL. This fixes the DPO logs in notebook environments.

```
from unsloth import PatchDPOTrainer
PatchDPOTrainer()
```

4. We can now import the other libraries. The main difference between DPO and SFT is the import of `DPOConfig` and `DPOTrainer` from TRL, which are specific to DPO training.

```
import os
import torch
from datasets import load_dataset
from transformers import TrainingArguments, TextStreamer
from unsloth import FastLanguageModel, is_bfloat16_supportedfrom trl import DPOC
```

5. This step loads our fine-tuned model from *Chapter 5*. We use the same configuration with a `max_seq_length` of 2048. You can activate QLoRA by setting `load_in_4bit` to `True`. In the following, we will perform LoRA DPO fine-tuning for increased speed and quality.

```
max_seq_length = 2048
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="mlabonne/TwinLlama-3.1-8B",
    max_seq_length=max_seq_length,
    load_in_4bit=False,
)
```

6. Let's now prepare the model for PEFT with the LoRA configuration. We increase the rank (`r`) and `lora_alpha` from `32` (as it was in *Chapter 5*) to `64`. This will allow more expressive fine-tuning. We keep a dropout of `0` for speed and we target every linear module as per usual.

```
model = FastLanguageModel.get_peft_model(
    model,
    r=32,
    lora_alpha=32,
    lora_dropout=0,
    target_modules=["q_proj", "k_proj", "v_proj", "up_proj", "down_proj", "o_pro
)
```

7. We load the `llmtwin-dpo` dataset (training split), which contains our prompts, chosen, and rejected answers.

```
dataset = load_dataset("mlabonne/llmtwin-dpo", split="train")
```

8. The data preparation is significantly different from the SFT example in *Chapter 5*. Here, we have triples with a prompt, a chosen answer, and a rejected answer. In the `format_samples` function, we apply the Alpaca chat template to each individual message. Note that the instruction is the only one that requires the chat format: chosen and rejected answers only need to be concatenated with the **end of sentence (EOS)** token. Finally, we create a train/test split with a 95%/5% ratio.

```
alpaca_template = """Below is an instruction that describes a task. Write a resp
### Instruction:
{}
### Response:
"""
EOS_TOKEN = tokenizer.eos_token
def format_samples(example):
    example["prompt"] = alpaca_template.format(example["prompt"])
    example["chosen"] = example['chosen'] + EOS_TOKEN
    example["rejected"] = example['rejected'] + EOS_TOKEN
    return {"prompt": example["prompt"], "chosen": example["chosen"], "rejected"
dataset = dataset.map(format_samples)
dataset = dataset.train_test_split(test_size=0.05)
```

9. The model and data are now ready, so we can start fine-tuning. Compared to SFT, there are a few new parameters, like `ref_model` and `beta`. Since we're using LoRA (or QLoRA), we don't directly train the model but instead the adapters. This means we can use the original model (without adapters) as a reference, saving a lot of VRAM. The `beta` parameter controls the importance of the reference model. A standard value of 0.1 works well in most scenarios, but we decided to increase it to 0.5 based on our experiments. This is due to the fact that the trained model used formal language with lower values. Having it closer to the reference model helps to fix this issue.

The learning rate is also lower (from 3e-4 for SFT to 2e-6 here). We train for 1 epoch instead of 3, and the `max_seq_length` parameter is now broken down into two new parameters: `max_prompt_length` (prompt only) and `max_length` (prompt and answer). Note that we also replaced the `TrainingArguments` class with `DPOConfig`.

```
trainer = DPOTrainer(
    model=model,
    ref_model=None,
    tokenizer=tokenizer,
    beta=0.5,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    max_length=max_seq_length//2,
    max_prompt_length=max_seq_length//2,
    args=DPOConfig(
        learning_rate=2e-6,
        lr_scheduler_type="linear",
        per_device_train_batch_size=2,
        per_device_eval_batch_size=2,
        gradient_accumulation_steps=8,
        num_train_epochs=1,
        fp16=not is_bfloat16_supported(),
        bf16=is_bfloat16_supported(),
        optim="adamw_8bit",
        weight_decay=0.01,
        warmup_steps=10,
```

```
            output_dir="output",
            eval_strategy="steps",
            eval_steps=0.2,
            logging_steps=1,
            report_to="comet_ml",
            seed=0,
        ),
    )
    trainer.train()
```

10. Once the model is trained, we can run it for a quick sanity check. This step is similar to the SFT example. It prepares the model for inference and generates a response to a prompt.

```
FastLanguageModel.for_inference(model)
message = alpaca_template.format("Write a paragraph to introduce supervised fine
inputs = tokenizer([message], return_tensors="pt").to("cuda")
text_streamer = TextStreamer(tokenizer)
_ = model.generate(**inputs, streamer=text_streamer, max_new_tokens=256, use_cac
```

11. The trained DPO model returns the following response:

```
Supervised fine-tuning is a method used to enhance the performance of pre-traine
```

We can compare it with the answer provided by the SFT model:

```
Supervised fine-tuning is a method used to enhance a language model by providing it
```

The DPO model provides an answer that is both more accurate and closer to the desired writing style. It correctly identifies pre-training language models as source models for SFT. It also mentions domain or task-specific finetunes instead of alignment with "human expectations," which is closer to the preference alignment stage. The answer is also less formal and something we would use in a blog post.

12. Finally, the last step consists of saving the trained model locally and pushing it to the Hugging Face Hub.

```
model.save_pretrained_merged("model", tokenizer, save_method="merged_16bit")
```

Congratulations! We have trained and exported our DPO model. It is now available on the Hugging Face Hub at https://huggingface.co/mlabonne/TwinLlama-3.1-8B-DPO. Compared to SFT, DPO has a few additional metrics that need to be tracked during training. *Figure 6.6* shows the Comet ML dashboard with the main metrics. You can publicly access it using the following URL: https://www.comet.com/mlabonne/llm-twin-training/

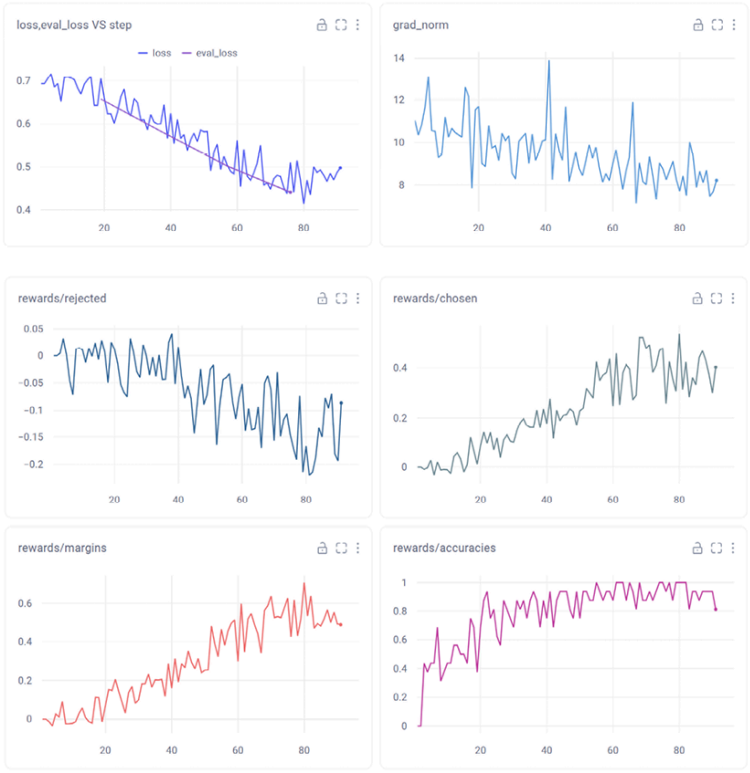


Figure 6.6 – Experiment tracking in Comet ML with DPO metrics

Let's review these metrics:

- **Training loss**: We still want the loss to continuously decrease on average. Note that it can rapidly fall to zero, meaning that the model is no longer learning anything. This behavior doesn't necessarily lead to overfitting or bad models but needs to be monitored closely.
- **Validation loss**: The same thing can be said about the validation loss. We expect a small gap compared to the training loss.
- **Gradient norm**: We expect small gradient norms with few spikes.
- **Rewards**: We have two different rewards: chosen and rejected. They correspond to the mean difference between the log probabilities output by the trained and reference models. Over time, we expect the model to choose the chosen answers and reject the rejected answers, which means that the gap between them should increase. This difference is directly tracked by the `margins` metric, defined as the difference between chosen and rejected rewards. A well-trained model's margin will quickly increase and then plateau.
- **Accuracies**: This metric represents the percentage of times the model correctly identifies the chosen answers. We want this accuracy to gradually increase during training, but it doesn't need to reach 100%. An accuracy of 100%, especially if it's achieved quickly, indicates that the preference dataset might be too easy for the model. While the LLM can still learn from such a dataset, it might be beneficial to add more challenging examples.

In general, DPO is slightly harder to monitor and debug than SFT because it's a more complex process, involving a reference model. However, it's also significantly easier to use than PPO and other RLHF algorithms. As long as you have a high-quality preference dataset and a strong fine-tuned model, you can experiment with different ranks, beta parameters, learning rates, and number of epochs to see which experiment best captures your preferences.

While this is not the purpose of this chapter, it is possible to automate the evaluation of models designed to imitate a writing style. A possible solution consists of comparing the distribution of words in the text generated by different models (SFT and DPO) with our ground-truth dataset. In this example, we expect the SFT model to output a lot of words that are over-represented in GPT-4o-mini (like "delve into"). The distribution output by our DPO model should be a lot closer to the chosen answers.

Summary

This chapter explored preference alignment techniques for improving LLMs. It introduced the concept of preference datasets, explaining their structure and importance in capturing nuanced human preferences. We implemented our own custom preference data generation pipeline by comparing original and AI-generated text from real articles. This pipeline can be reused and customized based on your use case.

We also provided an overview of the evolution of RLHF, leading to the introduction of DPO as a simpler and more efficient alternative. Finally, we implemented DPO using the Unsloth library to fine-tune our TwinLlama-3.1-8B model from *Chapter 5*. Our step-by-step tutorial gave practical instructions for training the model, as well as highlighting key differences from SFT. The final model is available on the Hugging Face Hub.

In the next chapter, we will explore the crucial topic of LLM evaluation, addressing the challenges and current approaches in assessing LLM performance. We'll cover the creation of domain-specific evaluation sets, examine why evaluation remains a persistent problem in the field, and introduce the concept of using larger models to evaluate smaller ones (LLM-as-a-judge). The chapter will conclude with a comprehensive evaluation pipeline, providing a structured framework for consistent and effective LLM evaluation.

References

- Rafael Rafailov et al.. "*Direct Preference Optimization: Your Language Model is Secretly a Reward Model*." arXiv preprint arXiv:2305.18290, May 2023.
- Timo Kaufmann et al.. "*A Survey of Reinforcement Learning from Human Feedback*." arXiv preprint arXiv:2312.14925, December 2023.
- Anthropic. "*GitHub - anthropics/hh-rlhf: Human preference data for "Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback"*." github.com, 2022, https://github.com/anthropics/hh-rlhf.
- Nisan Stiennon et al.. "*Learning to summarize from human feedback*." arXiv preprint arXiv:2009.01325, September 2020.
- Intel(R) Neural Compressor. "Supervised Fine-Tuning and Direct Preference Optimization on Intel Gaudi2." medium.com, March 26, 2024, https://medium.com/intel-analytics-software/the-practice-of-supervised-finetuning-and-direct-preference-optimization-on-habana-gaudi2-a1197d8a3cd3.
- Argilla. "*GitHub - argilla-io/distilabel*." github.com, August 23, 2024, https://github.com/argilla-io/distilabel.
- Databricks. "*Enhancing LLM-as-a-Judge with Grading Notes*." databricks.com, July 22, 2024, https://www.databricks.com/blog/enhancing-llm-as-a-judge-with-grading-notes.

- Akrour, Riad & Schoenauer, Marc & Sebag, Michèle. (2011). Preference-Based Policy Learning. 12-27. 10.1007/978-3-642-23780-5_11.
- Cheng, Weiwei & Fürnkranz, Johannes & Hüllermeier, Eyke & Park, Sang-Hyeun. (2011). *Preference-Based Policy Iteration: Leveraging Preference Learning for Reinforcement Learning*. 312-327. 10.1007/978-3-642-23780-5_30.
- Paul Christiano et al.. "*Deep reinforcement learning from human preferences*." arXiv preprint arXiv:1706.03741, June 2017.
- Long Ouyang et al.. "*Training language models to follow instructions with human feedback*." arXiv preprint arXiv:2203.02155, March 2022.
- John Schulman et al.. "*Proximal Policy Optimization Algorithms*." arXiv preprint arXiv:1707.06347, July 2017.
- unslothai. "*GitHub - unslothai/unsloth: Finetune Llama 3.1, Mistral*, Phi & Gemma LLMs 2-5x faster with 80% less memory." github.com, August 21, 2024, https://github.com/unslothai/unsloth.

Join our book's Discord space

Join our community's Discord space for discussions with the authors and other readers:

https://packt.link/llmeng

Answers collapsed